Universidad Nacional de San Agustín

Escuela Profesional de Ciencia de la Computación

Laboratorio A: Programación con Pthreads

Reducciones, histograma y sincronización

Nombre:
David Tamo Turpo

Curso:
Computación Paralela y Distribuida

Repositorio GitHub:
github.com/davichiacustico-ux/laboratorio-a-pthreads

Profesor:
Alvaro Henry Mamani Aliaga

1. Entorno de ejecución, métricas y scripts de prueba

1.1 Entorno de ejecución

Tabla 1: Equipo donde se ejecutaron las tres partes (de `entorno_p1.txt`, `entorno_p2.txt` y `entorno_p3.txt`).

Componente	Valor
Procesador	Intel(R) Core(TM) i5-10400 CPU @ 2.90GHz
Núcleos e hilos	6 núcleos físicos × 2 hilos por núcleo = 12 CPU lógicas
Memoria visible en Linux	11 GiB
Sistema (uname)	Linux 6.18.54.1-microsoft-standard-WSL2
Compilador	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, opciones `-O2 -pthread`
Mediciones de la Parte I	Thu Oct 29 02:32:57 −05 2026
Mediciones de la Parte II	Thu Oct 29 02:33:11 −05 2026
Mediciones de la Parte III	Thu Oct 29 02:34:25 −05 2026

Las tres partes se midieron en el mismo equipo y en la misma sesión. El procesador tiene 6 núcleos físicos con *hyper-threading* (12 CPU lógicas): con T = 8 hay más threads que núcleos físicos, así que algunos núcleos ejecutan dos threads que comparten sus unidades de cálculo y su caché. Las pruebas corren en Linux dentro de WSL2, una máquina virtual ligera sobre Windows 10 Pro; la memoria visible es la asignada a esa máquina virtual, y crear un thread o despertar a uno que duerme cuesta más que en Linux nativo (secciones 2.3 y 4.3).

1.2 Métricas y forma de medir

$$S_T = \frac{T_s}{T_p(T)} \qquad E_T = \frac{S_T}{T}$$

donde T_s es el tiempo de la versión secuencial, $T_p(T)$ el de la versión paralela con T threads, S_T el *speedup* y E_T la eficiencia.

- **Qué se mide.** En las versiones paralelas el cronómetro incluye crear los threads, esperarlos con `pthread_join` y combinar los resultados parciales, es decir, el costo completo de paralelizar. La generación de los datos no se mide; usa una semilla fija (`srand(42)`) para que todas las versiones reciban exactamente el mismo arreglo.
- **Reloj.** `clock_gettime(CLOCK_MONOTONIC)`, que no salta si se ajusta la hora del sistema. Los tiempos se imprimen con resolución de nanosegundos (`%.9f`).
- **Repeticiones.** Cada programa ejecuta el cálculo una vez de calentamiento (`r = -1`), que no se guarda, y luego 7 veces medidas; se reporta la **mediana** y, como medida de dispersión, la medición mínima y la máxima. La mediana no se deja arrastrar por una ejecución interrumpida por el sistema operativo: con N $\geq 10^7$ la diferencia entre la medición más lenta y la más rápida fue típicamente de 9 a 11 %, pero con N pequeños llegó a 77 %.
- **Threads.** T = 1, 2, 4 y 8, como pide el enunciado.

1.3 Compilación y explicación de los scripts (`.sh`)

El flujo de pruebas de cada parte está automatizado en un script de shell POSIX (`experimentos_p1.sh`, `experimentos_p2.sh` y `experimentos_p3.sh`) que se ejecuta con:

```
sh experimentos_p1.sh   # datos_p1.csv, resultados_p1.csv, repartos.txt, speedup,
eficiencia
sh experimentos_p2.sh   # datos_p2.csv, resultados_p2*.csv, speedup_p2, eficiencia_p2,
distribucion_p2
sh experimentos_p3.sh   # datos_p3.csv, resultados_p3.csv, tiempos_p3,
cambios_contexto_p3.csv
```

Cada script `.sh` sigue cinco pasos internos:

1. **Formato numérico uniforme (`export LC_ALL=C`):** Fuerza el uso de punto decimal (`.`) en `awk` y en la salida estándar aunque el sistema esté configurado en español, evitando que una coma decimal (`0,054`) rompa las columnas de los archivos CSV.

2. **Compilación verificada (`gcc -O2 -pthread ... || exit 1`):** Compila los programas secuenciales y paralelos con el mismo nivel de optimización (`-O2`). El operador `|| exit 1` detiene inmediatamente el script si `gcc` devuelve algún error.

3. **Captura automática del hardware (`entorno_pN.txt`):** Un bloque agrupado `{ date; uname -sr; lscpu | grep -E ...; free -h; gcc --version | head -n 1; } > entorno_pN.txt` registra la fecha de inicio, kernel/WSL2, modelo de CPU, núcleos físicos e hilos lógicos, memoria RAM y versión de `gcc`.

4. **Extracción de métricas a CSV (`guardar_fila` con `awk`):** Los programas en C imprimen cada resultado en una línea con formato `Clave = Valor` (por ejemplo, `Suma = 489014` o `Tiempo paralelo (mediana de 7) = 0.000060700 s`). En el script, la función `guardar_fila()` captura la salida del programa y la procesa con `awk`:

   ```
   guardar_fila() {
       VALORES=$(echo "$4" | awk -F'= ' '/^(Suma|Minimo|Maximo|Promedio|Tiempo)/ {sub(/ s$/,
   "", $2); printf ",%s", $2}')
       echo "$1,$2,$3$VALORES" >> datos_p1.csv
   }
   ```

   `awk -F'= '` divide cada línea por el separador `=`, filtra únicamente las líneas de interés mediante una expresión regular, elimina la unidad `" s"` al final de los tiempos con `sub(/ s$/, "", $2)` y concatena los valores separados por comas para escribir una fila completa en `datos_pN.csv`.

5. **Barrido de combinaciones y llamada a Python:**

   - En **`experimentos_p1.sh`**: recorre `VALORES_N="1000 100000 1000000 1000003 10000000 100000000"`. Por cada `N`, ejecuta una vez `./p1sec "$N"` y luego barre `VALORES_T="1 2 4 8"` ejecutando `./p1par "$N" "$T"`. De la misma salida de `p1par` filtra con `grep '^Thread [0-9]'` los rangos `[inicio, fin)` asignados a cada hilo y los guarda en `repartos.txt`. Finalmente ejecuta `python3 analizar_p1.py`.
   - En **`experimentos_p2.sh`**: define la función auxiliar `experimento()`, que para una distribución (`u` o `s`) y un tamaño `N` corre primero `./p2sec "$N" "$dist"` y después, dentro de un bucle `for T in 1 2 4 8`,

ejecuta las tres variantes paralelas (`./p2mutex`, `./p2mutexcat` y `./p2priv`). El script llama a `experimento u "$N"` para los cinco tamaños (`1000` a `10000000`) y luego a `experimento s 10000000` para evaluar los datos sesgados. Al terminar invoca `python3 analizar_p2.py`.

- En **`experimentos_p3.sh`**: recorre `BUFFERS="8 64 1024"` con `TOTAL=100003`. Para cada tamaño de buffer ejecuta `./p3sec "$TOTAL" "$B"` y luego recorre `PARES_PC="1:1 2:2 4:4 8:8 2:4 4:2"`, separando productores y consumidores mediante expansión de parámetros POSIX (`P=${PC%:*}` extrae lo que está antes de `:` y `C=${PC#*:}` lo que está después) para invocar `./p3par "$TOTAL" "$P" "$C" "$B"`. Al final ejecuta `python3 analizar_p3.py` y `python3 cambios_contexto_p3.py > cambios_contexto_p3.csv`.

Cada programa también se puede compilar y ejecutar manualmente por separado:

```
gcc -O2 -pthread p1sec.c -o p1sec            # ./p1sec N
gcc -O2 -pthread p1par.c -o p1par            # ./p1par N T
gcc -O2 -pthread p2sec.c -o p2sec            # ./p2sec N [u|s]
gcc -O2 -pthread p2mutex.c -o p2mutex        # ./p2mutex N T [u|s]
gcc -O2 -pthread p2mutexcat.c -o p2mutexcat  # ./p2mutexcat N T [u|s]
gcc -O2 -pthread p2priv.c -o p2priv          # ./p2priv N T [u|s]
gcc -O2 -pthread p3sec.c -o p3sec            # ./p3sec TOTAL TAM_BUFFER
gcc -O2 -pthread p3par.c -o p3par            # ./p3par TOTAL P C TAM_BUFFER
```

El argumento opcional `u|s` de la Parte II elige datos uniformes (por defecto) o sesgados (sección 3.4).

2. Estadísticas del conjunto de datos (Parte I)

2.1 Estrategia de paralelización

El arreglo `datos[0..N-1]`, con valores entre 0 y 999, se divide en T bloques contiguos. Con $b = \lfloor N/T \rfloor$ y $r = N \bmod T$, el thread `id` recibe el intervalo [inicio, fin) con

$$\text{inicio} = \text{id} \cdot b + \min(\text{id}, r), \qquad \text{fin} = \text{inicio} + b + [\,\text{id} < r\,]$$

Es decir, los primeros r threads procesan $b + 1$ elementos y los demás b: la carga difiere a lo sumo en un elemento y ningún índice queda sin procesar ni se procesa dos veces.

Tabla 2: Reparto de N = 1 000 003 elementos entre T = 8 threads (de `repartos.txt`): 1 000 003 = 8 × 125 000 + 3, así que los threads 0, 1 y 2 reciben un elemento más.

Thread	Inicio	Fin (excluido)	Elementos
0	0	125 001	125 001
1	125 001	250 002	125 001
2	250 002	375 003	125 001
3	375 003	500 003	125 000
4	500 003	625 003	125 000
5	625 003	750 003	125 000
6	750 003	875 003	125 000
7	875 003	1 000 003	125 000

Cada thread recorre su bloque acumulando en **variables locales** la suma, el mínimo y el máximo. La suma es `long long` porque con N = 10^8 llega a unos 5×10^{10} y no cabe en un `int`. Al terminar, cada thread escribe una sola vez sus tres resultados en su propia estructura `args[id]`. El hilo principal espera a todos con `pthread_join` y combina los T resultados parciales: suma de las sumas, mínimo de los mínimos y máximo de los máximos; el promedio es la suma global dividida entre N. No se usa ningún mutex.

2.2 Comprobación de resultados

Tabla 3: Resultados de la Parte I. Para cada N, las cuatro ejecuciones paralelas dieron exactamente la misma suma, mínimo, máximo y promedio que la versión secuencial.

N	Suma	Mínimo	Máximo	Promedio	T probados	¿Igual al secuencial?
1 000	489 014	0	999	489.0140	1, 2, 4, 8	OK
100 000	50 026 152	0	999	500.2615	1, 2, 4, 8	OK
1 000 000	499 639 820	0	999	499.6398	1, 2, 4, 8	OK
1 000 003	499 641 123	0	999	499.6396	1, 2, 4, 8	OK
10 000 000	4 995 813 121	0	999	499.5813	1, 2, 4, 8	OK
100 000 000	49 954 148 838	0	999	499.5415	1, 2, 4, 8	OK

2.3 Tiempos, speedup y eficiencia

Tabla 4: Tiempos de la Parte I: mediana de 7 mediciones (Tp incluye crear, esperar y combinar los threads) y rango de las 7 mediciones paralelas.

N	T	Ts (ms)	Tp (ms)	Tp mín – máx (ms)	Speedup	Eficiencia
1 000	1	0.000606	0.0607	0.0516 – 0.0703	0.01	0.01
	2		0.0920	0.0818 – 0.143	0.01	0.00
	4		0.128	0.127 – 0.148	0.01	0.00
	8		0.363	0.321 – 0.428	0.00	0.00
100 000	1	0.0567	0.123	0.111 – 0.133	0.46	0.46
	2		0.146	0.0936 – 0.165	0.39	0.19
	4		0.146	0.140 – 0.198	0.39	0.10
	8		0.301	0.290 – 0.317	0.19	0.02
1 000 000	1	0.649	0.784	0.757 – 1.16	0.83	0.83
	2		0.701	0.617 – 0.859	0.93	0.46
	4		0.400	0.377 – 0.459	1.62	0.41
	8		0.425	0.368 – 0.533	1.53	0.19
1 000 003	1	0.723	0.705	0.660 – 0.767	1.03	1.03
	2		0.454	0.388 – 0.638	1.59	0.80
	4		0.393	0.323 – 0.422	1.84	0.46
	8		0.454	0.406 – 0.540	1.59	0.20
10 000 000	1	6.70	7.01	6.82 – 7.59	0.95	0.95
	2		3.90	3.73 – 6.13	1.72	0.86
	4		2.22	2.11 – 2.27	3.02	0.76
	8		2.07	1.99 – 2.21	3.23	0.40
100 000 000	1	67.6	68.2	66.8 – 68.7	0.99	0.99
	2		34.5	34.3 – 38.7	1.96	0.98
	4		24.1	19.8 – 28.6	2.81	0.70
	8		18.9	18.0 – 19.5	3.58	0.45

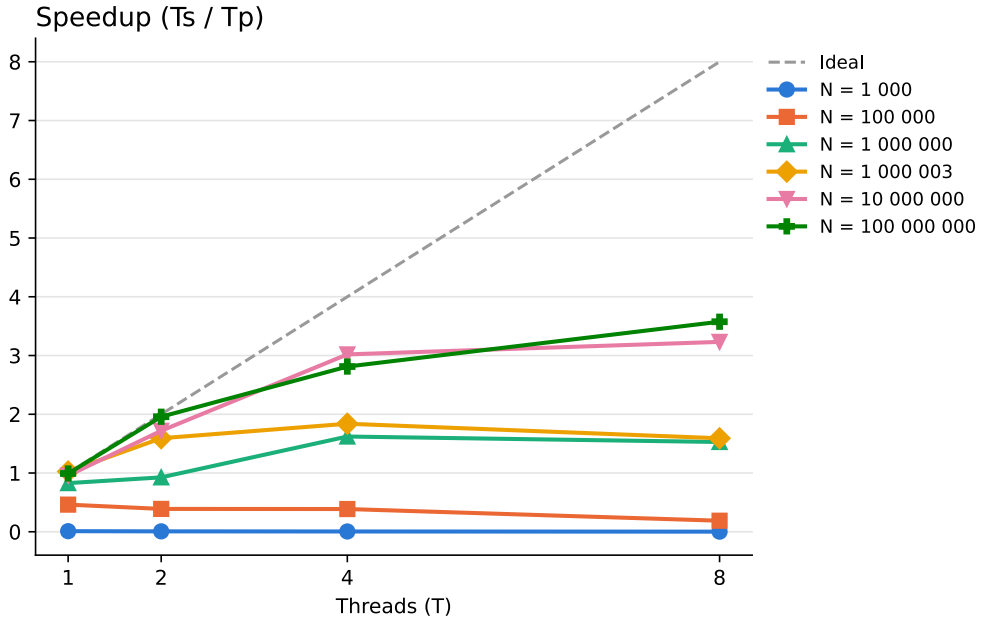


Figura 1: Speedup de la Parte I. La línea gris punteada es el speedup ideal (igual a T).

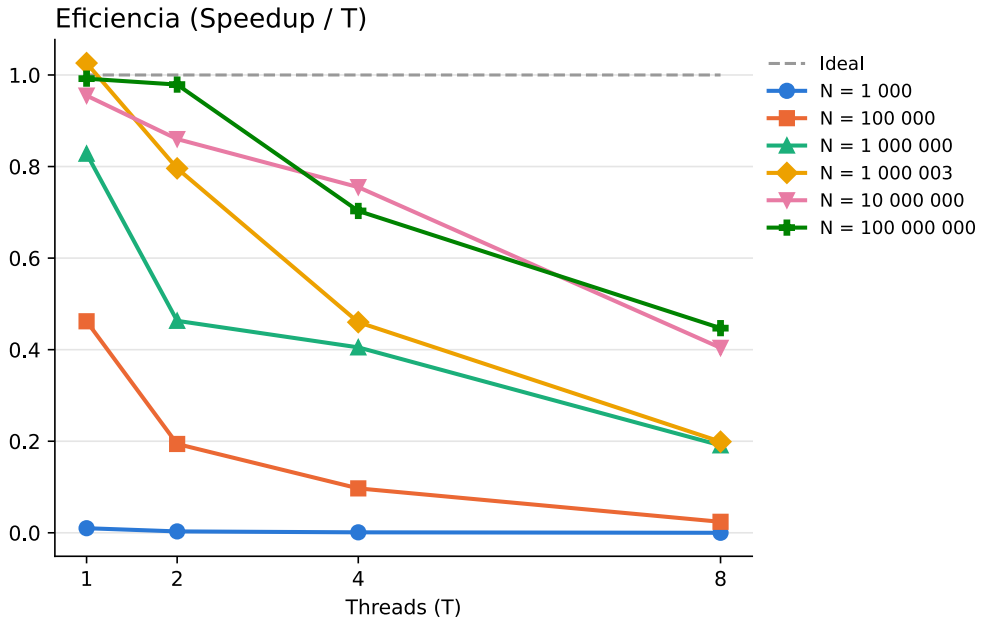


Figura 2: Eficiencia de la Parte I. La línea gris punteada es la eficiencia ideal (1).

- **Con N pequeños (10^3 y 10^5) el paralelo pierde.** Recorrer 1 000 elementos toma 0.6 µs, mientras que crear y esperar un solo thread cuesta unos 60 µs en este equipo, y 8 threads unos 360 µs (diferencia $T_p - T_s$ con N = 1 000). Con N = 10^5 el cálculo secuencial dura 57 µs, todavía menos que ese costo fijo, y el speedup no pasa de 0.46.
- **Con N = 10^6 la ganancia es moderada.** Con unos 0.7 ms de trabajo, el costo fijo ya es una fracción menor y se llega a un speedup de 1.6 a 1.8 con 4 threads, pero pasar a 8 no ayuda: cada thread tiene tan poco trabajo que el costo de crearlo vuelve a pesar.
- **Con N grandes (10^7 y 10^8) el paralelo gana, pero lejos del ideal.** Con N = 10^8, dos threads logran $S_2 = 1.96$ (eficiencia 0.98, casi perfecta), cuatro $S_4 = 2.81$ y ocho $S_8 = 3.58$. Hay dos frenos. Primero, el cálculo está **limitado por la memoria**: por cada entero de 4 bytes leído solo se hace una suma y dos comparaciones. Un thread lee los 400 MB del arreglo en 67.6 ms (unos 5.9 GB/s); con 8 threads la lectura sube a unos 21 GB/s y la memoria pasa a ser el recurso compartido que limita a todos los núcleos. Segundo, con T = 8 hay más threads que núcleos físicos.

- **N = 1 000 003**, que no es múltiplo de 2, 4 ni 8, se comporta igual que N = 10^6: el elemento extra de algunos threads no desbalancea la carga.
- **Con T = 1** y N ≥ 10^6, el speedup queda entre 0.83 y 1.03: la versión paralela hace el mismo trabajo más la creación de un thread, y por debajo de 1 ms la medición varía hasta un 68 %.

2.4 Respuestas a las preguntas de análisis

¿Por qué conviene calcular resultados parciales en cada thread? Porque así cada thread trabaja sin compartir nada durante el recorrido: sus acumuladores locales viven en registros del procesador, no hay que sincronizar en cada elemento y ningún núcleo invalida la caché de otro. La única comunicación es una escritura por thread al final y una combinación de T valores en el hilo principal, un costo O(T) despreciable frente al O(N/T) del recorrido.

¿Qué problema surgiría si todos los threads actualizaran una única variable de suma sin sincronización? Una condición de carrera. `suma += datos[i]` no es atómica: el procesador lee `suma`, le suma el valor y la escribe. Si dos threads leen el mismo valor antes de que el otro escriba, una de las dos actualizaciones se pierde (*lost update*): la suma final saldría menor que la real y distinta en cada ejecución. Y protegerla con un mutex daría un resultado correcto pero lento: es lo que mide el histograma con mutex global de la Parte II, 21 veces más lento que el secuencial incluso con un solo thread (sección 3.3).

¿Es necesario utilizar un mutex para combinar los resultados parciales? Justifique su estrategia. No, con esta estrategia. Cada thread escribe solo en su propia estructura `args[id]`, y el hilo principal lee esas estructuras después de `pthread_join`. `pthread_join` es un punto de sincronización: garantiza que el thread terminó y que todo lo que escribió es visible para quien lo esperó. Durante la combinación hay un solo thread trabajando, así que no hay nada que proteger. Haría falta un mutex si cada thread sumara su resultado a una variable global compartida antes de terminar.

¿Cómo se distribuye el trabajo cuando N no es múltiplo del número de threads? Con el reparto por bloques de la sección 2.1: los primeros $N \bmod T$ threads reciben $\lfloor N/T \rfloor + 1$ elementos y el resto $\lfloor N/T \rfloor$. Para N = 1 000 003 y T = 8, tres threads procesan 125 001 elementos y cinco procesan 125 000. Los resultados coincidieron con el secuencial en todos los casos.

¿Por qué el speedup puede ser menor que el número de threads? Por varias razones que se suman. (1) El costo fijo de crear y esperar threads, de 60 µs con 1 thread a 360 µs con 8 en este equipo, que domina con N pequeños. (2) El ancho de banda de memoria: el cálculo es tan simple que, con varios threads, la memoria no alcanza a entregar los datos al ritmo al que los núcleos los procesan. (3) Los recursos compartidos: con T = 8 hay más threads que núcleos físicos, y con todos los núcleos ocupados el procesador no puede mantener la frecuencia turbo de un solo núcleo. (4) La parte secuencial (repartir el arreglo, combinar los resultados) no se acelera (ley de Amdahl).

3. Construcción del histograma (Parte II)

3.1 Estrategias implementadas

Cada valor $v \in [0, 99]$ va a la categoría $c = \lfloor v/10 \rfloor \in \{0, \ldots, 9\}$ (en C, `datos[i] / 10` con división entera). Se compararon cuatro programas que procesan el mismo arreglo con el mismo reparto en bloques de la Parte I:

Tabla 5: Las cuatro versiones del histograma.

Programa	Histograma	Sincronización durante el conteo
`p2sec`	Uno, local (`hist[c]++`)	Ninguna: un solo thread
`p2mutex`	Uno, compartido (`hist_global[10]`)	Un mutex global: `lock` y `unlock` por cada elemento
`p2mutexcat`	Uno, compartido; cada casilla con su contador y su mutex, en su propia línea de caché de 64 bytes	Un mutex por categoría: solo compiten threads que cuentan la misma categoría
`p2priv`	Uno privado por thread (`local[10]`)	Ninguna; el hilo principal suma los T histogramas tras `pthread_join` (10 × T sumas)

`p2mutexcat` se agregó para estudiar con mediciones la pregunta sobre la distribución de los datos. Para eso, todos los programas aceptan un argumento opcional: `u` genera datos uniformes (por defecto) y `s` datos sesgados, donde 9 de cada 10 valores se generan entre 0 y 9, de modo que cerca del 91 % de los datos cae en la categoría 0.

3.2 Comprobación de resultados

En las 24 combinaciones medidas (20 con datos uniformes y 4 con datos sesgados), las 10 categorías de las tres versiones paralelas coincidieron exactamente con las de la versión secuencial con los mismos datos, y su suma fue exactamente N.

Tabla 6: Histograma secuencial de N = 10 000 000 valores con las dos distribuciones. Las tres versiones paralelas obtuvieron los mismos conteos con T = 1, 2, 4 y 8.

Categoría	Intervalo	Uniforme	Sesgada
c0	00–09	1 000 011	9 100 248
c1	10–19	1 000 434	99 533
c2	20–29	1 000 793	99 737
c3	30–39	999 584	99 665
c4	40–49	1 000 287	100 155
c5	50–59	1 000 361	100 146
c6	60–69	999 878	100 018
c7	70–79	1 000 923	100 165
c8	80–89	998 621	100 415
c9	90–99	999 108	99 918
Total		**10 000 000**	**10 000 000**

3.3 Rendimiento con datos uniformes

Tabla 7: Histograma con datos uniformes. Tiempos en ms (mediana de 7); S = speedup, E = eficiencia; priv. = histograma privado, global = un mutex global, categ. = un mutex por categoría. En todas las filas las 10 categorías coinciden con el secuencial y suman N.

N	T	Ts	Tp priv.	S priv.	E priv.	Tp global	S global	Tp categ.	S categ.
1 000	1	0.000801	0.0594	0.01	0.01	0.0823	0.010	0.0771	0.010
	2		0.0863	0.01	0.01	0.0857	0.009	0.0944	0.008
	4		0.138	0.01	0.00	0.135	0.006	0.138	0.006
	8		0.278	0.00	0.00	0.279	0.003	0.282	0.003
100 000	1	0.0804	0.152	0.53	0.53	1.90	0.042	1.73	0.046
	2		0.130	0.62	0.31	8.53	0.009	3.70	0.022
	4		0.154	0.52	0.13	5.31	0.015	3.38	0.024
	8		0.305	0.26	0.03	7.19	0.011	3.27	0.025
1 000 000	1	0.895	0.986	0.91	0.91	18.6	0.048	16.9	0.053
	2		0.543	1.65	0.82	87.5	0.010	32.2	0.028
	4		0.387	2.31	0.58	46.2	0.019	34.7	0.026
	8		0.434	2.06	0.26	66.6	0.013	32.7	0.027
1 000 003	1	0.840	1.03	0.81	0.81	18.5	0.046	16.8	0.050
	2		0.582	1.44	0.72	87.3	0.010	31.4	0.027
	4		0.374	2.25	0.56	47.4	0.018	33.8	0.025
	8		0.392	2.14	0.27	62.0	0.014	29.9	0.028
10 000 000	1	8.60	9.54	0.90	0.90	182	0.047	166	0.052
	2		5.19	1.66	0.83	805	0.011	337	0.026
	4		2.83	3.04	0.76	476	0.018	340	0.025
	8		2.58	3.33	0.42	666	0.013	325	0.026

- **El histograma privado escala como la Parte I.** Con N = 10^7 llega a $S_4 = 3.04$ y $S_8 = 3.33$. Con N pequeños, igual que en la Parte I, domina el costo de crear los threads.
- **El mutex global es siempre más lento que el secuencial.** Con un solo thread ya tarda 181.8 ms frente a los 8.6 ms del secuencial (21 veces más): bloquear y desbloquear un mutex libre cuesta unos 17 ns por elemento, unas 20 veces lo que cuesta contarlo. Con 2 threads el tiempo sube a 805 ms (94 veces el secuencial): los threads se turnan el candado en cada elemento y la línea de caché del mutex y del histograma viaja de un núcleo a otro millones de veces. Con 4 y 8 threads (476 y 666 ms) no mejora, porque el conteo queda completamente serializado: el speedup no pasa de 0.05 en ninguna configuración.
- **El mutex por categoría alivia, pero no resuelve.** Con datos uniformes, repartir los candados baja el tiempo de 805 a 337 ms con T = 2 (2.4 veces menos que el mutex global) y de 666 a 325 ms con T = 8, pero sigue siendo entre 38 y 40 veces más lento que el secuencial: el `lock` y `unlock` por elemento y el viaje de las líneas de caché entre núcleos se mantienen.
- Con N = 10^7 y T = 8, el histograma privado (2.58 ms) es **258 veces más rápido** que el mutex global y 126 veces más rápido que el mutex por categoría.

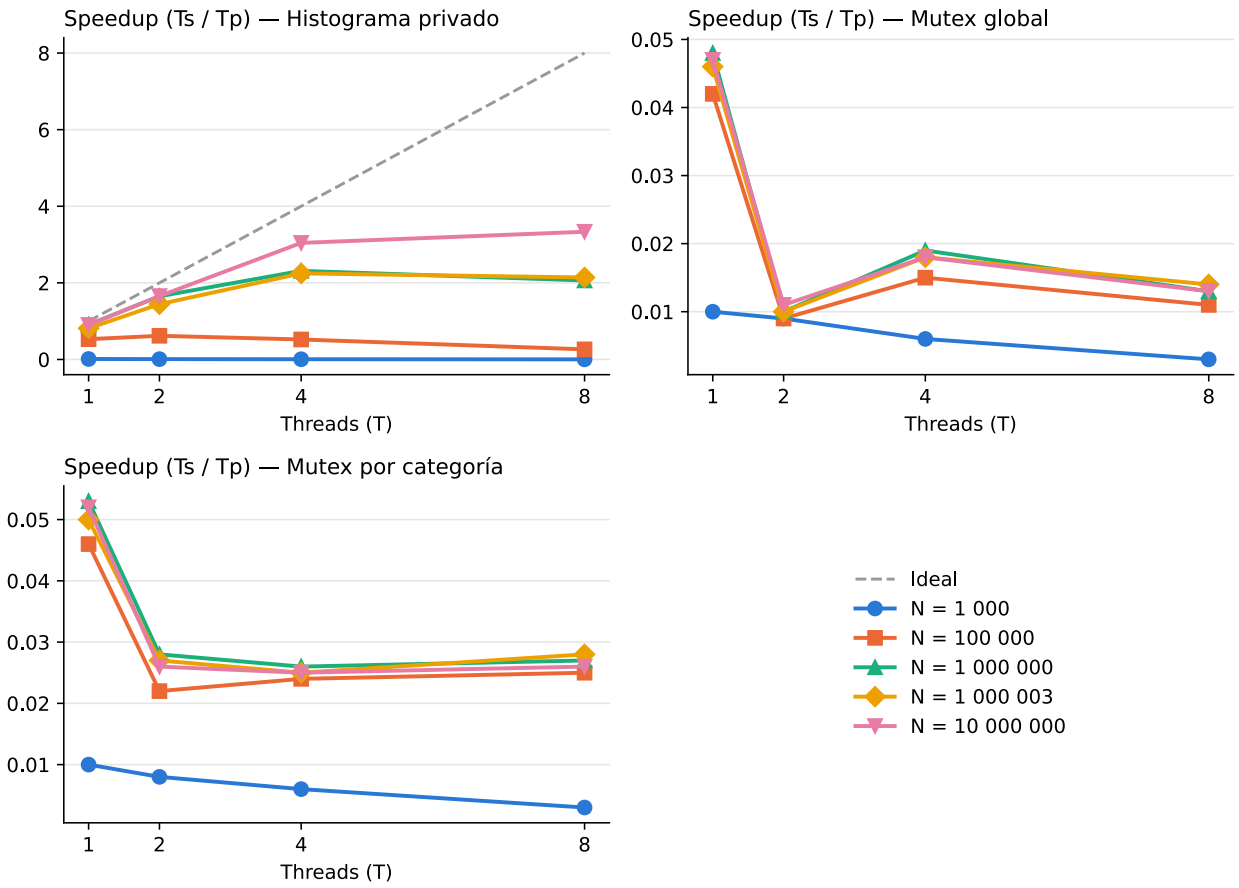


Figura 3: Speedup de las tres estrategias con datos uniformes. Cada panel tiene su propia escala vertical: las dos versiones con mutex no llegan a 0.06.

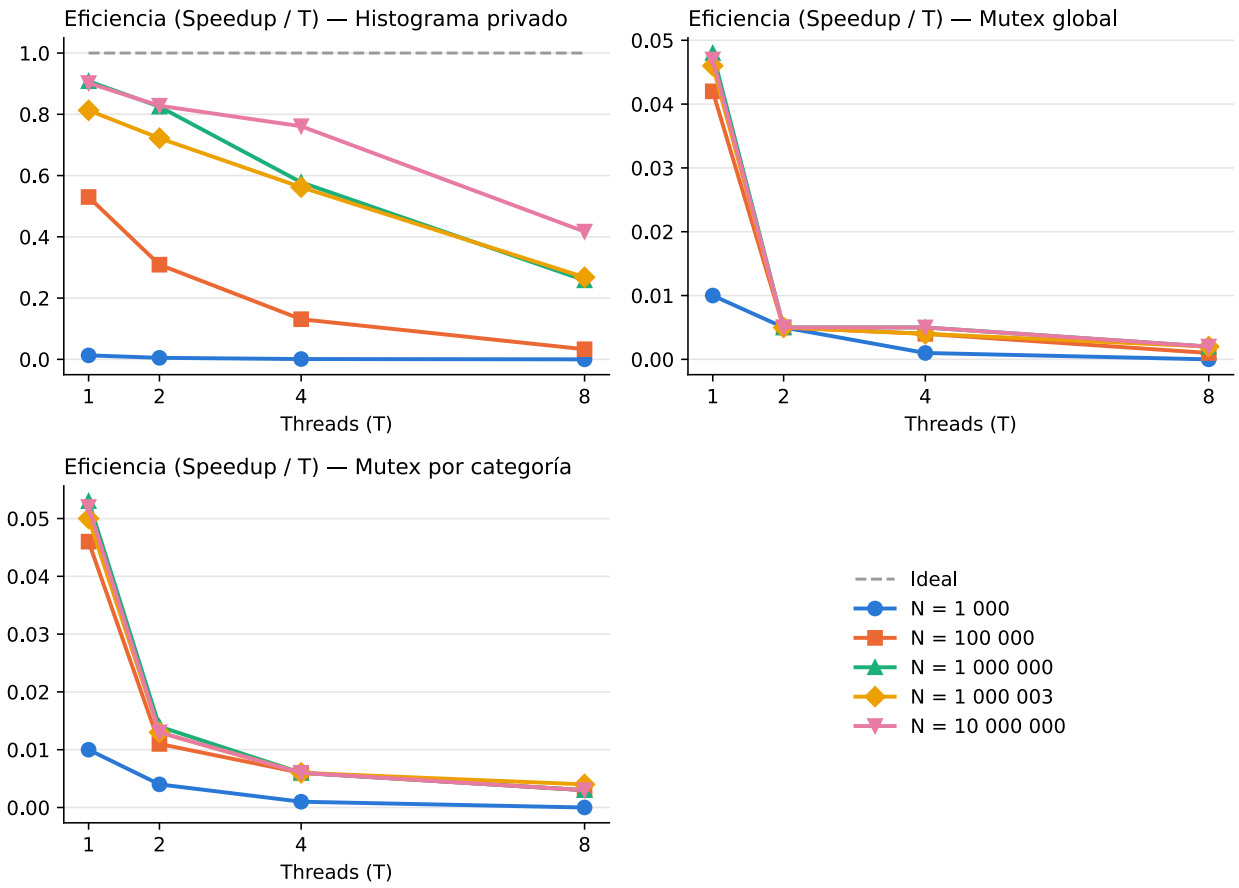


Figura 4: Eficiencia de las tres estrategias con datos uniformes.

3.4 Influencia de la distribución de los datos

Tabla 8: Influencia de la distribución de los datos (N = 10 000 000, tiempos en ms, mediana de 7). La última columna compara el mutex por categoría con el global: menos de 1 significa que repartir los candados ayudó.

Datos	T	Ts	Tp priv.	Tp global	Tp categ.	categ. / global
Uniforme	1	8.60	9.54	182	166	0.91
	2		5.19	805	337	0.42
	4		2.83	476	340	0.71
	8		2.58	666	325	0.49
Sesgada	1	13.2	13.4	184	166	0.90
	2		6.85	812	603	0.74
	4		3.56	473	522	1.10
	8		2.74	664	637	0.96

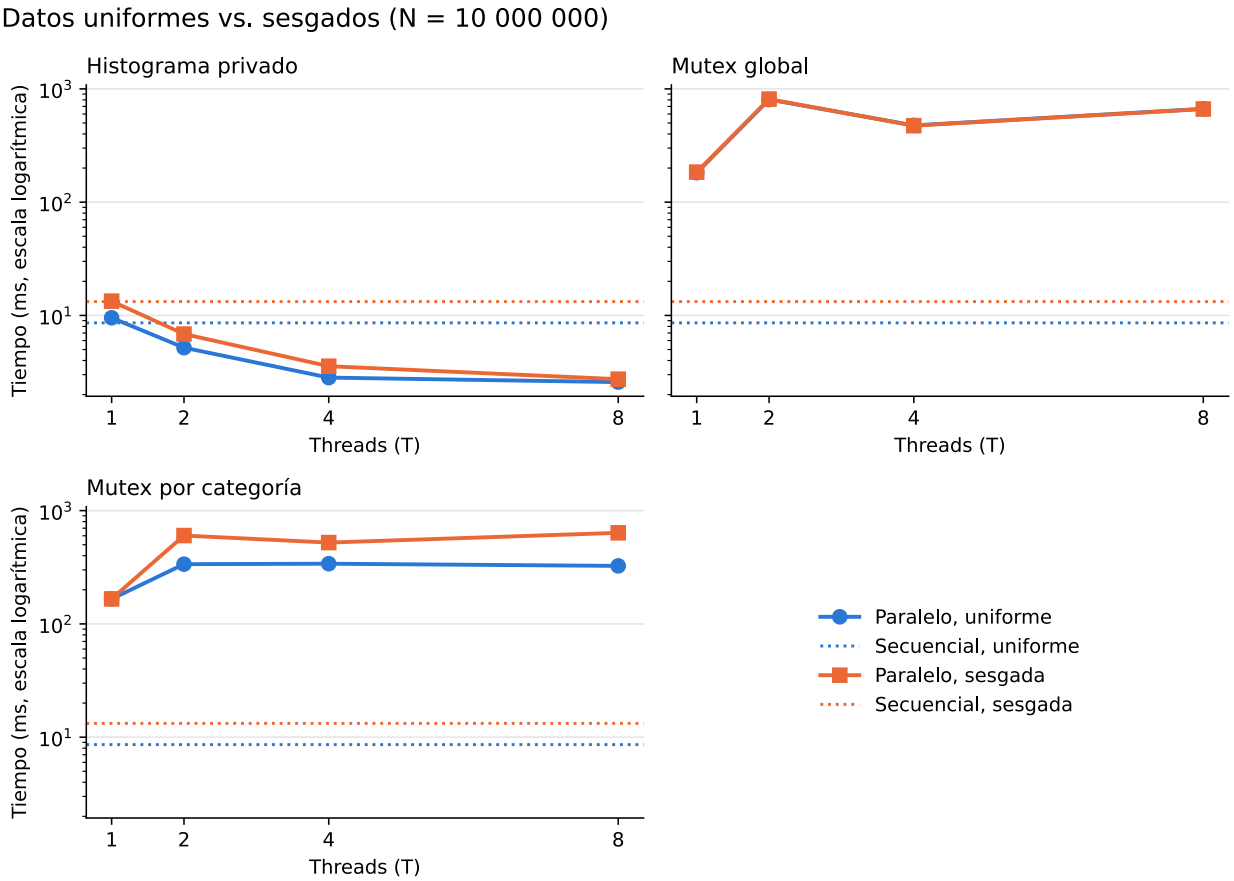


Figura 5: Tiempos con datos uniformes y sesgados (N = 10^7, escala logarítmica). Las líneas punteadas son el secuencial con los mismos datos.

- **Con un mutex global, la distribución no importa.** Los tiempos con datos uniformes y sesgados difieren menos de un 2 %: todos los threads compiten por el mismo candado, caigan sus valores donde caigan.
- **Con un mutex por categoría, la distribución decide.** Con datos sesgados, cerca del 91 % de los incrementos piden el candado de la categoría 0 y el mutex por categoría se comporta como un mutex global: con T = 2, 4 y 8 tarda 603, 522 y 637 ms, entre 1.5 y 2 veces más que con datos uniformes, y su tiempo queda entre 0.74 y 1.10 veces el del mutex global, frente a 0.42 a 0.71 veces con datos uniformes.
- **Con histograma privado no hay contención, pero los datos sesgados encarecen cada incremento.** El secuencial y el privado con un thread son entre 1.4 y 1.5 veces más lentos con datos sesgados (el secuencial, 13.2 frente a 8.6 ms): cuando casi todos los valores caen en la misma casilla, cada `hist[0]++` debe esperar a que termine el anterior (lectura, suma y escritura sobre la misma dirección de memoria), y esa cadena de dependencias impide que el procesador solape incrementos consecutivos. Ese costo sí se reparte entre los threads, porque cada uno tiene su propio contador: con 8 threads el privado baja a 2.74 ms, un speedup de 4.84 respecto del secuencial con los mismos datos.

3.5 Respuestas a las preguntas de análisis

¿En qué situación se produce una condición de carrera al actualizar el histograma? Cuando dos o más threads incrementan a la vez la misma casilla de un histograma compartido sin exclusión mutua. `hist[c]++` es leer, sumar y escribir: si dos threads leen el mismo valor de `hist[c]` antes de que alguno escriba, ambos escriben el valor más uno y se pierde un conteo, de modo que la suma de las categorías queda por debajo de N. Con datos uniformes ocurre cuando dos threads procesan al mismo tiempo valores del mismo intervalo; con datos sesgados, casi siempre.

¿Qué diferencias hay entre usar un mutex compartido y un histograma privado por thread? Ambos dan el resultado correcto, pero el mutex compartido sincroniza en cada uno de los N elementos: cada `lock` y `unlock` cuesta tiempo aunque nadie compita, solo un thread puede contar a la vez y el mutex y el histograma

viajan constantemente entre las cachés de los núcleos. El histograma privado no sincroniza durante el conteo: cada thread cuenta en su propio arreglo de 10 contadores (80 bytes, un costo de memoria despreciable) dentro de su caché, y la combinación final son solo 10 × T sumas después de `pthread_join`. Medido con N = 10^7 y T = 8: 666 ms frente a 2.58 ms.

¿Qué estrategia esperarías que escale mejor y por qué? El histograma privado: su costo es O(N/T) sin esperas más O(10 T) para combinar. Con un mutex compartido cada elemento es una sección crítica, así que el conteo se ejecuta de uno en uno y agregar threads solo agrega contención. Los datos lo confirman: el privado llega a $S_8 = 3.33$, mientras que el mutex global no pasa de 0.05 y empeora al pasar de 1 a 2 threads.

¿Cómo influye la distribución de los valores de entrada en la contención? Depende de la estrategia (sección 3.4). Con un mutex global, nada: la contención es máxima siempre. Con un mutex por categoría, mucho: con datos uniformes los threads se reparten entre 10 candados y chocan poco, y con datos sesgados casi todos piden el mismo candado y el tiempo se acerca al del mutex global. Con histogramas privados la distribución no genera contención, aunque los datos sesgados vuelven más lento cada incremento, también en el secuencial, por la dependencia entre incrementos a la misma casilla.

4. Productores y consumidores (Parte III)

4.1 Diseño del buffer y de la sincronización

El estado compartido es `buffer[TAM_BUFFER]` (arreglo circular), `in` (próxima posición donde insertar), `out` (próxima posición de donde extraer) y `cuenta` (elementos almacenados, entre 0 y `TAM_BUFFER`). Los índices avanzan con `(i + 1) % TAM_BUFFER`, así que el arreglo se recorre en círculo y se conserva el orden de llegada. Un mutex, `candado`, protege todo ese estado, y dos variables de condición coordinan las esperas:

- `no_lleno`: los productores esperan en ella mientras `cuenta == TAM_BUFFER`;
- `no_vacio`: los consumidores esperan en ella mientras `cuenta == 0`.

```
pthread_mutex_lock(&candado);
while (cuenta == TAM_BUFFER)                // buffer lleno: dormir
    pthread_cond_wait(&no_lleno, &candado); // suelta el candado mientras duerme
buffer[in] = (int) item;                    // insertar
in = (in + 1) % TAM_BUFFER;
cuenta++;
pthread_cond_signal(&no_vacio);             // despierta a un consumidor
pthread_mutex_unlock(&candado);
```

El consumidor es simétrico: espera en `no_vacio` mientras `cuenta == 0`, extrae de `buffer[out]` y avisa en `no_lleno`. La condición se vuelve a evaluar con `while` y no con `if` porque, entre el `signal` y el momento en que el thread despertado recupera el candado, otro thread del mismo tipo puede haberse llevado el elemento (o el hueco); además, POSIX admite despertares espurios. Como hay una variable de condición para cada tipo de thread, un `signal` nunca despierta a un thread del tipo equivocado.

Los TOTAL elementos son los enteros 0, 1, …, TOTAL − 1, repartidos en bloques entre los P productores (el mismo reparto de la Parte I). Cada consumidor extrae una cuota fija, también repartida en bloques; como las cuotas suman exactamente TOTAL, ningún consumidor espera un elemento que nunca llegará.

4.2 Verificación

Como cada elemento es un entero distinto, la verificación es exacta. En cada ejecución se comprueba que:

1. Producidos = Consumidos = TOTAL y el buffer termina vacío (`cuenta == 0`).
2. La suma de lo producido y la de lo consumido valen $0 + 1 + \ldots + (\text{TOTAL} - 1) = \text{TOTAL}(\text{TOTAL} - 1)/2 = 5\,000\,250\,003$.
3. Cada consumidor hace `vistos[item]++` dentro de la sección crítica; al final todas las casillas de `vistos[0..TOTAL-1]` deben valer exactamente 1. Una casilla en 0 es un elemento perdido y una mayor que 1, un duplicado.

Cada configuración se ejecuta 8 veces (1 de calentamiento y 7 medidas) y cada ejecución se verifica por separado. En las 18 configuraciones (144 ejecuciones) hubo 0 elementos perdidos, 0 duplicados y 0 ejecuciones fallidas. También se probaron casos extremos con buffer de una sola posición (8 productores y 1 consumidor, 1 productor y 8 consumidores) y con menos elementos que threads (5 elementos, 8 productores y 8 consumidores), sin errores.

4.3 Resultados

Tabla 9: Productores y consumidores con TOTAL = 100 003 elementos. Tiempos: mediana de 7 mediciones. Perdidos, duplicados y ejecuciones fallidas suman las 8 ejecuciones de cada configuración (1 de calentamiento + 7 medidas).

Buffer	P–C	Ts (ms)	Tp (ms)	Tp mín – máx (ms)	Tp / Ts	Perdidos	Duplicados	Ejec. fallidas	Estado
8	1P–1C	0.753	445	443 – 448	590	0	0	0	OK
	2P–2C		1901	1868 – 1910	2523	0	0	0	OK
	4P–4C		684	678 – 695	907	0	0	0	OK
	8P–8C		718	710 – 724	953	0	0	0	OK
	2P–4C		746	728 – 774	990	0	0	0	OK
	4P–2C		771	737 – 803	1023	0	0	0	OK
64	1P–1C	1.35	55.1	53.1 – 56.0	41	0	0	0	OK
	2P–2C		760	661 – 865	561	0	0	0	OK
	4P–4C		122	117 – 133	90	0	0	0	OK
	8P–8C		128	121 – 137	94	0	0	0	OK
	2P–4C		188	140 – 209	139	0	0	0	OK
	4P–2C		171	163 – 211	126	0	0	0	OK
1024	1P–1C	1.43	17.5	15.3 – 18.8	12	0	0	0	OK
	2P–2C		58.5	36.6 – 66.6	41	0	0	0	OK
	4P–4C		19.6	18.7 – 26.6	14	0	0	0	OK
	8P–8C		25.2	24.4 – 28.9	18	0	0	0	OK
	2P–4C		58.5	38.6 – 76.7	41	0	0	0	OK
	4P–2C		58.7	47.9 – 77.4	41	0	0	0	OK

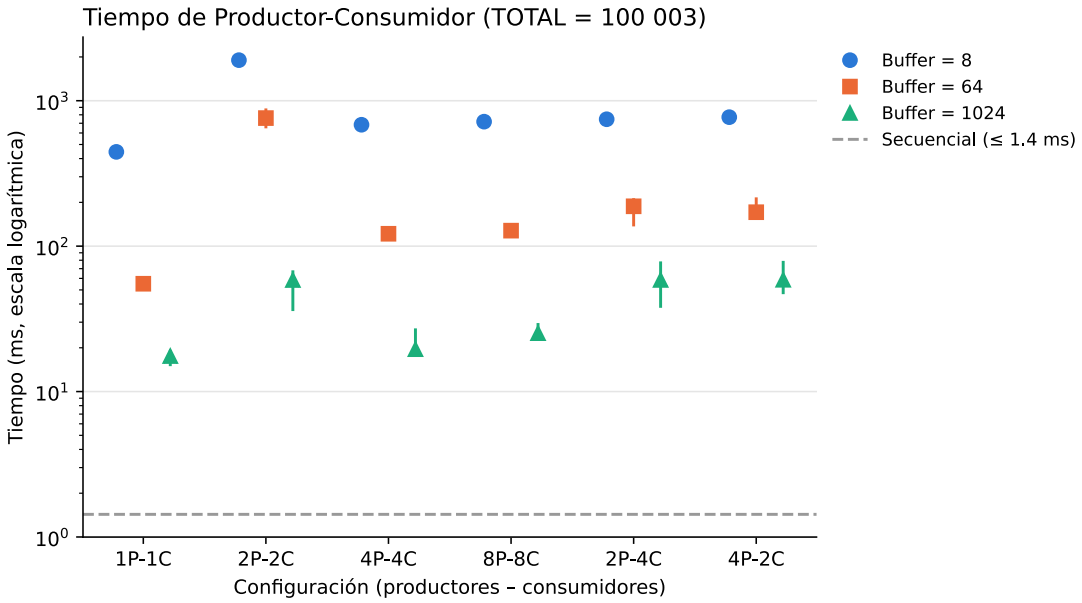


Figura 6: Tiempo de cada configuración: mediana de 7 mediciones; la barra vertical va de la medición más rápida a la más lenta. Escala logarítmica.

Tabla 10: Esperas por ejecución: cambios de contexto voluntarios, es decir, cuántas veces se durmió algún thread en el mutex o en una variable de condición (sumando todos los threads, medido con `getrusage` por `cambios_contexto_p3.py`), y tiempo de esa misma corrida. B = tamaño del buffer.

P–C	Esperas (B = 8)	ms (B = 8)	Esperas (B = 64)	ms (B = 64)	Esperas (B = 1024)	ms (B = 1024)
1P–1C	25 050	444	3 021	53.5	78	18.7
2P–2C	172 441	1892	80 336	894	5 437	54.9
4P–4C	90 599	688	16 617	122	4 964	21.9
8P–8C	95 657	721	19 999	126	8 955	24.8
2P–4C	84 436	778	19 311	165	7 612	58.2
4P–2C	84 716	773	19 209	170	7 498	68.9

- **El paralelo es mucho más lento que el secuencial, y es lo esperable.** El secuencial produce y consume los 100 003 enteros en 0.75 a 1.43 ms; la mejor configuración paralela tarda 17.5 ms (12 veces más) y la peor 1.9 s (unas 2 500 veces más). Cada elemento cuesta un `lock` y un `unlock`, y cada vez que el buffer se llena o se vacía un thread se duerme y otro debe despertarlo, lo que en este equipo cuesta del orden de 10 µs. Producir un entero no cuesta casi nada, así que no hay cálculo que repartir: el objetivo de esta parte es la correctitud de la sincronización. Con productores y consumidores que hicieran trabajo real (leer un archivo, procesar cada dato), el buffer permitiría solapar ambas etapas.
- **El tamaño del buffer es lo que más influye.** Con 8 posiciones el buffer se llena y se vacía constantemente y las configuraciones tardan de 445 a 1 901 ms; con 64, de 55 a 760 ms, y con 1024, de 17.5 a 58.7 ms, entre 13 y 35 veces menos que con 8. Un buffer grande absorbe las ráfagas de productores y consumidores, y los threads se duermen muchas menos veces: con 1P–1C, de unas 25 000 veces por ejecución con buffer 8 a menos de 100 con buffer 1024.
- **2P–2C es la configuración más lenta con buffer 8 y 64.** No es ruido: se repite en las 7 mediciones (dispersión menor al 10 % con buffer 8) y en una segunda corrida independiente. Con 2P–2C los threads se duermen unas 172 000 veces por ejecución con buffer 8, casi el doble que con 4P–4C y 7 veces más que con 1P–1C; con buffer 8 y 64, el tiempo sigue de cerca a ese número. Una explicación compatible con los datos son los despertares inútiles: el thread despertado por `signal` encuentra que otro de su mismo tipo ya tomó el elemento (o el hueco) y vuelve a dormirse, justamente el caso que obliga a usar `while`.

- Con 8P–8C hay 16 threads para 12 CPU lógicas, y es la única configuración en la que el sistema desaloja threads involuntariamente (de 117 a 1 010 veces por ejecución).

4.4 Respuestas a las preguntas de análisis

¿Por qué no basta con proteger únicamente el contador de elementos? Porque insertar y extraer modifican tres datos acoplados: la casilla del buffer, el índice (`in` u `out`) y `cuenta`. Si solo `cuenta` estuviera protegido, dos productores podrían leer el mismo `in` y escribir en la misma casilla, y se perdería un elemento; dos consumidores podrían leer el mismo `out` y llevarse el mismo elemento, que quedaría duplicado. Un consumidor podría incluso ver `cuenta` incrementado antes de que el productor terminara de escribir la casilla. Toda la operación (comprobar la condición, leer o escribir la casilla, avanzar el índice y actualizar `cuenta`) debe ser una única sección crítica.

¿Qué puede ocurrir si un consumidor intenta extraer un elemento de un buffer vacío? Leería una casilla que no contiene un elemento nuevo: basura o un elemento ya consumido, que quedaría duplicado. Además `cuenta` quedaría negativo y `out` adelantaría a `in`, desordenando el buffer para todas las operaciones siguientes. Por eso el consumidor duerme en `no_vacio` mientras `cuenta == 0`.

¿Qué puede ocurrir si un productor inserta en un buffer lleno? Sobrescribiría un elemento que todavía no se consumió, que se perdería; `cuenta` superaría `TAM_BUFFER` e `in` pasaría por encima de `out`, rompiendo el orden de llegada. Por eso el productor duerme en `no_lleno` mientras `cuenta == TAM_BUFFER`.

¿Qué función cumplen el mutex y las variables de condición o semáforos elegidos? El mutex da exclusión mutua sobre `buffer`, `in`, `out` y `cuenta`. Las variables de condición permiten esperar sin consumir CPU: `pthread_cond_wait` suelta el mutex y duerme al thread en un solo paso atómico, de modo que no se pierde un aviso entre comprobar la condición y dormirse, y al despertar vuelve a tomar el mutex antes de continuar. `no_lleno` es la sala de espera de los productores y la avisa un consumidor al liberar un hueco; `no_vacio` es la de los consumidores y la avisa un productor al insertar. Con semáforos se lograría lo mismo con dos semáforos contadores (huecos libres y elementos disponibles) más un mutex.

¿Cómo comprobarías que todos los elementos producidos fueron consumidos exactamente una vez? Dando a cada elemento un identificador único (0 … TOTAL − 1) y registrando cada consumo con `vistos[item]++`, como en la sección 4.2: al final todas las casillas deben valer 1. Los conteos y las sumas son una comprobación adicional, pero por sí solos no bastan: perder el 2 y el 4 y consumir dos veces el 3 deja iguales el conteo y la suma ($2 + 4 = 3 + 3$), mientras que `vistos` detecta cualquier pérdida o duplicado.

5. Resultados y análisis general

1. **Los resultados se comprobaron de tres formas.** Por comparación con la versión secuencial (Partes I y II), por verificación exacta elemento por elemento en cada una de las 8 ejecuciones de cada configuración (Parte III) y con ThreadSanitizer (`gcc -fsanitize=thread`), que no reportó ninguna carrera de datos en `p1par`, `p2mutex`, `p2mutexcat`, `p2priv` ni `p3par`. Todos los programas compilan sin avisos con `-Wall -Wextra`.
2. **El costo fijo de los threads marca un tamaño mínimo.** Crear y esperar threads cuesta de 60 µs (1 thread) a 360 µs (8 threads) en este equipo. Con N por debajo de 10^6 el trabajo no alcanza a pagarlo (speedup menor que 1); desde 10^6 a 10^7 elementos, sí.
3. **La frecuencia de sincronización decide el rendimiento.** Las cuatro versiones del histograma hacen exactamente el mismo conteo y, con $N = 10^7$ y $T = 8$, tardan desde 2.58 ms (privado, sin sincronizar) hasta 666 ms (un mutex global por elemento). En la Parte III, el tiempo sigue al número de veces que los threads se duermen y se despiertan.
4. **Los límites del hardware aparecen antes que el ideal.** Con cálculos limitados por la memoria y con 8 threads en 6 núcleos físicos, el mejor speedup fue 3.58 (Parte I) y 3.33 (Parte II); la eficiencia baja de 0.98 con 2 threads a 0.45 con 8 (Parte I, $N = 10^8$).
5. **Medir bien cambia las conclusiones.** Las mediciones individuales variaron hasta un 77 % con N pequeños. En la primera versión de este laboratorio, con una sola medición por caso, el histograma privado con un thread parecía más rápido que el secuencial (eficiencia 1.28); con 7 mediciones y la mediana, ese resultado

desaparece (0.90). Una segunda corrida independiente de la Parte III reprodujo los tiempos con buffer 8 con diferencias menores al 5 %.

6. Problemas encontrados y soluciones

Tabla 11: Problemas encontrados durante el laboratorio y cómo se resolvieron.

Problema	Causa	Solución
Desbordamiento de la suma	Con N = 10^8 y valores hasta 999, la suma llega a unos 5×10^{10}, más que el máximo de un `int` (2 147 483 647)	`long long` en los acumuladores locales y en el global
N no divisible entre T	Si cada thread recibe `N / T` elementos, los `N % T` últimos quedan sin procesar	Reparto por bloques con un elemento extra para los primeros `N % T` threads
Histograma con mutex global muy lento	Un `lock` y `unlock` por elemento serializa a los threads y hace viajar la línea de caché del mutex entre núcleos	Histograma privado por thread; se agregó un mutex por categoría para medir el efecto de la distribución
Elemento robado entre el `signal` y el despertar; despertares espurios	Con `if`, el thread despertado no vuelve a comprobar la condición	Esperar siempre dentro de `while (cuenta == 0)` y `while (cuenta == TAM_BUFFER)`
Comas decimales en los CSV	En Linux configurado en español, `awk` imprime `0,054` y rompe las columnas	`export LC_ALL=C` en los scripts
Mediciones ruidosas	Una sola medición por caso: el histograma privado con un thread parecía más rápido que el secuencial	Una ejecución de calentamiento y 7 medidas; se reporta la mediana, la mínima y la máxima
Reloj y resolución	`CLOCK_REALTIME` puede saltar si se ajusta la hora; guardado en un `double` (unos 1.8 $\times 10^9$ s) solo distingue unos 0.24 µs, y los tiempos se imprimían con 6 decimales	`CLOCK_MONOTONIC`, que cuenta desde el arranque y conserva resolución de nanosegundos, e impresión con `%.9f`
Datos de dos equipos mezclados	La primera versión tenía la Parte I medida en el Core Ultra 9 285 del laboratorio y las Partes II y III en otro equipo, con un único `entorno.txt`	Se volvió a medir todo en un mismo equipo y cada parte guarda su `entorno_pN.txt`; la versión anterior se conserva en `version_anterior/`
Dos juegos de scripts	`run_p*.sh` y `experimentos_p*.sh` escribían el mismo `resultados_p*.csv` con columnas distintas	Un solo flujo: `experimentos_p*.sh` más `analizar_p*.py`
Fallos silenciosos	No se revisaba el resultado de `pthread_create`, y con TOTAL mayor que `INT_MAX` los identificadores `int` de la Parte III se desbordaban	`crear_thread()` termina el programa si no se puede crear un thread, y se valida TOTAL

7. Conclusiones

1. **Comprobación de resultados.** Las tres partes produjeron resultados idénticos a sus versiones secuenciales en todas las configuraciones: las mismas estadísticas para los seis tamaños de arreglo, incluido N = 1 000 003, que no es múltiplo de 2, 4 ni 8; histogramas con las 10 categorías iguales y suma exactamente N, con datos uniformes y sesgados; y 0 elementos perdidos o duplicados en las 144 ejecuciones de productores y consumidores. ThreadSanitizer no detectó carreras de datos.
2. **Sincronización.** En reducciones e histogramas, la mejor sincronización es no necesitarla durante el cálculo: resultados parciales privados y una combinación después de `pthread_join`. Un mutex por elemento es correcto pero, con N = 10^7, entre 21 y 94 veces más lento que el secuencial; repartirlo por categorías ayuda solo si los datos también se reparten, y con datos sesgados vuelve a comportarse como un mutex global. Cuando los threads deben comunicarse mientras trabajan, como en el buffer acotado, un mutex con dos variables de condición evaluadas en `while` garantiza la exclusión mutua y la espera sin consumo de CPU.
3. **Rendimiento.** El paralelismo compensa solo cuando el trabajo supera el costo de crear threads (N de al menos 10^6 en este equipo) y la sincronización es poco frecuente. Se alcanzó un speedup de 3.58 con 8 threads

en la Parte I ($N = 10^8$) y de 3.33 con el histograma privado ($N = 10^7$), limitados por el ancho de banda de memoria y por tener más threads que núcleos físicos. En productores y consumidores, el tamaño del buffer fue el factor dominante: pasar de 8 a 1024 posiciones redujo el tiempo entre 13 y 35 veces, porque los threads se duermen y se despiertan muchas menos veces.

Anexo A. Archivos del laboratorio y repositorio

Repositorio público con todo el código, scripts, mediciones, figuras y comparativas: `https://github.com/davichiacustico-ux/laboratorio-a-pthreads`

Tabla 12: Archivos entregados en el repositorio.

Archivo	Contenido
`p1sec.c`, `p1par.c`	Parte I: estadísticas, versión secuencial y paralela
`p2sec.c`, `p2mutex.c`, `p2mutexcat.c`, `p2priv.c`	Parte II: histograma secuencial, con mutex global, con un mutex por categoría y privado por thread
`p3sec.c`, `p3par.c`	Parte III: buffer circular secuencial y con productores y consumidores
`experimentos_p1.sh`, `experimentos_p2.sh`, `experimentos_p3.sh`	Compilan, registran el equipo (`entorno_pN.txt`), ejecutan todas las combinaciones y llaman al análisis
`analizar_p1.py`, `analizar_p2.py`, `analizar_p3.py`	Verifican los resultados, calculan speedup y eficiencia y dibujan los gráficos
`cambios_contexto_p3.py`	Cuenta cuántas veces se duermen los threads de la Parte III
`datos_p*.csv`, `repartos.txt`, `cambios_contexto_p3.csv`	Mediciones en bruto
`resultados_p*.csv`	Mediciones verificadas, con speedup y eficiencia
`figuras/*.svg` y `*.png`	Gráficos (SVG para este informe, PNG para verlos rápido)
`informe/`	Fuente de este informe; `informe/tablas.py` genera todas sus tablas desde los CSV
`Laboratorio_A_Explicado.md`, `explicacion_experimentos_p*.md`	Material de estudio con explicaciones paso a paso
`prueba_sin_calentamiento/`	Experimento comparativo sin vuelta de calentamiento (`r = 0..6` y 1.ª vuelta en frío vs. `r = -1`)
`version_anterior/`	Respaldo de la primera versión (mediciones de dos equipos y de una sola repetición)